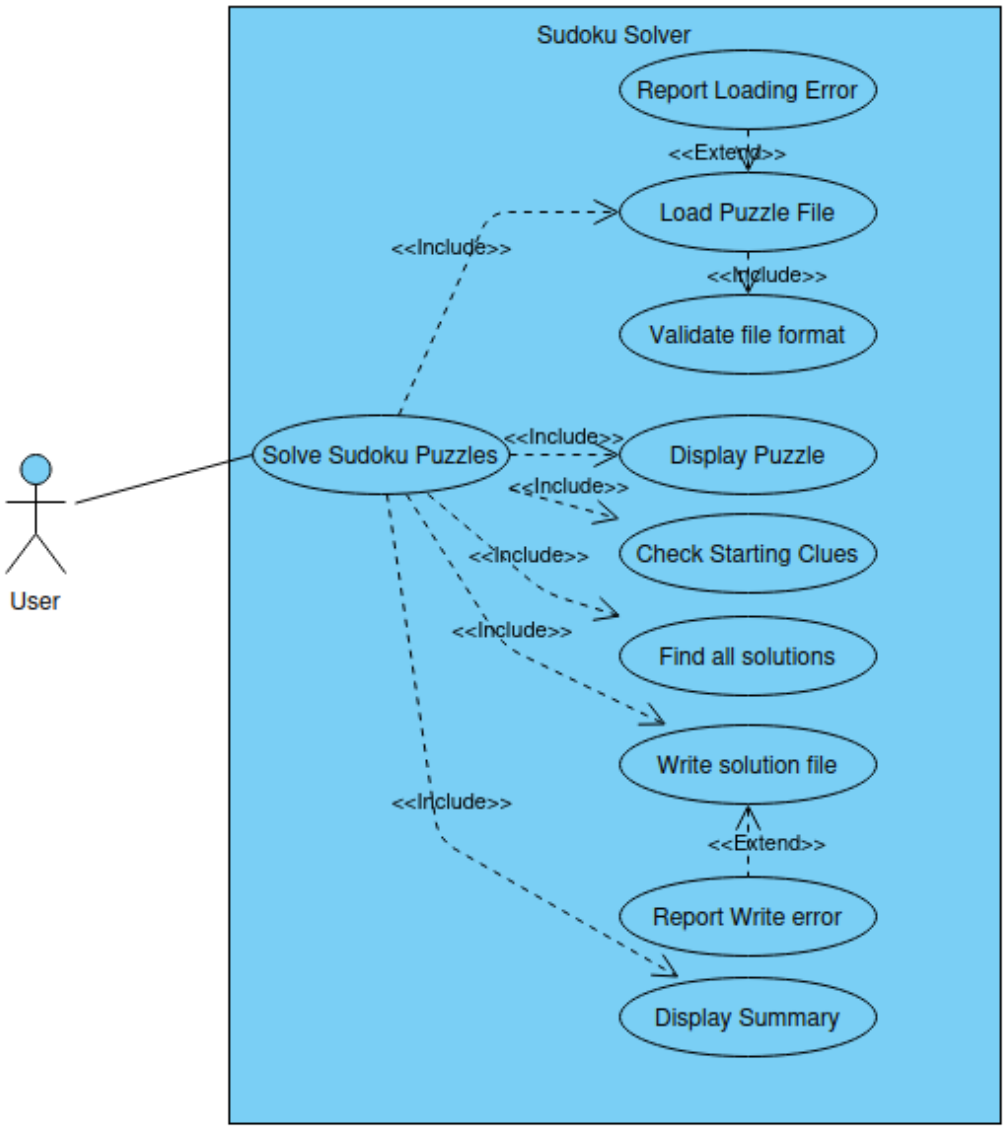


a)

Primary Actor: User

Main Use Case: Solve Puzzle Files

b)

The user runs the program, which processes puzzle1.txt through puzzle5.txt in the working directory. For each file, the system reads the grid from the file system, validates the file

format, displays the original puzzle, checks that the starting clues don't conflict, and finds every valid solution. Each solution is written to the matching solutionN.txt, followed by a solution count, or by "No solution found." if none exists. The terminal shows the filename, the original puzzle, and a short summary. If a puzzle file can't be loaded or a solution file can't be written, the system reports the error and moves on to the next file.

c)

The User starts the program with puzzle1.txt through puzzle5.txt in the working directory.

For each puzzle number, the system displays the filename and loads the file from the File System.

The system validates the format: exactly 81 cells, each a digit 1 through 9 or an underscore, with whitespace allowed.

The system displays the original puzzle, using underscores for empty cells.

The system checks the starting clues for row, column, and block conflicts.

The system opens solutionN.txt for writing on the File System.

The system takes the next empty cell in reading order and tries each digit that causes no conflict, recursing after each placement. If no digit fits, it backtracks.

When all 81 cells are filled, the system writes that solution to solutionN.txt, adds one to the solution count, and keeps searching for others.

When the search is exhausted, the system writes "Solutions found: n", or "No solution found." if the count is zero, and closes the file.

The system displays a summary on the terminal with the solution count and the name of the results file.

The system moves on to the next file. After the fifth, the program ends.